

Projet : Mise en place d'un intranet en haute disponibilité avec Docker

1. Objectif

L'objectif de ce projet est de mettre en place une infrastructure intranet hautement disponible permettant d'assurer la continuité de service, même en cas de panne d'un serveur. Cette solution vise également à améliorer les performances grâce à la répartition de charge, tout en optimisant l'utilisation des ressources de la machine hôte.

2. Architecture de la solution

L'infrastructure repose sur plusieurs conteneurs Docker, chacun ayant un rôle spécifique.

2.1 Serveurs web

Trois conteneurs **Nginx** ont été déployés.

Chaque conteneur héberge une copie du site web et fonctionne de manière indépendante.

Chaque serveur web utilise un volume persistant afin de stocker les fichiers du site. Cela permet de conserver les données même en cas de redémarrage ou de suppression du conteneur.

2.2 Répartition de charge (Load Balancing)

Un conteneur **HAProxy** est utilisé comme répartiteur de charge.

Son rôle est de distribuer les requêtes entrantes entre les trois serveurs Nginx. Cette répartition permet :

- Améliorer les performances globales
- Éviter la surcharge d'un seul serveur
- Assurer une tolérance aux pannes (si un serveur tombe, les autres prennent le relais)

3. Gestion des mises à jour et synchronisation des fichiers

3.1 Principe

Le contenu du site web est centralisé sur la machine hôte dans le répertoire suivant :
/var/www/html

Ce contenu est ensuite synchronisé automatiquement vers les conteneurs Nginx.

3.2 Méthode utilisée

Une tâche planifiée (cron) est configurée pour s'exécuter toutes les minutes.

Cette tâche utilise l'outil **rsync** afin de copier les fichiers du répertoire source vers les volumes persistants associés à chaque conteneur.

3.3 Avantages de cette approche

- Mise à jour automatique et régulière du site
- Aucune interruption de service pendant les modifications
- Pas besoin de redéployer les conteneurs

- Possibilité de modifier le site en arrière-plan sans impacter les utilisateurs

Mise en place de l'environnement réseau et infrastructure

1. Configuration du VLAN sur le switch Cisco

- Création du VLAN 60 nommé **docker** sur le switch Cisco.

	VLAN ID	Name	Status
☐	1	default	active
☐	10	PRODUCTION	active
☐	20	ADMIN	active
☐	30	CLIENTS	active
☐	40	Sauvegarde	active
☐	50	SIEM	active
☐			

2. Configuration du pare-feu Stormshield

- Ajout de l'objet réseau correspondant au VLAN 60 dans Stormshield.

RÉSEAU / INTERFACES

Entrer un filtre...

Interface | Port | Type

in_vlan10_prod | 2 | Ethernet

in_vlan20_admin | 2 | VLAN, Id

in_vlan30_client | 2 | VLAN, Id

in_vlan50_siem | 2 | VLAN, Id

in_vlan40_sauve | 2 | VLAN, Id

in_vlan60_dock | 2 | VLAN, Id

out | 1 | Ethernet

dmz1 | 3 | Ethernet

CONFIGURATION DE IN_VLAN60_DOCK

CONFIGURATION GÉNÉRALE | CONFIGURATION AVANCÉE

Paramètres généraux

Nom: in_vlan60_dock

Commentaire:

Interface parente: in

Identifiant: 60

Priorité (CoS): 0

Cette interface est: ☒ Interne (protégée) ☐ Externe (publique)

Plan d'adressage

Adressage: ☐ Plan d'adressage hérité du bridge ☒ Dynamique / Statique

Adresse IPv4: ☐ IP dynamique (obtenue par DHCP) ☒ IP fixe (statique)

Ajouter **Supprimer**

Adresse / Masque	Commentaire
192.168.4.246/29	

- Mise en place des règles de filtrage et NAT spécifiques au vlan60

11	☒	on	passer	ServeurDocker	Internet	Any	FW
12	☒	on	passer	Network_in_vlan50_siem Network_in_vlan40_sauve	Internet	http https ftp	FW
13	☒	on	passer	Network_in_vlan50_siem Network_in_vlan40_sauve	Any	Any icmp	FW
14	☒	on	passer	Network_in_vlan50_siem Network_in_vlan40_sauve	Serveur_NTP_Lyce	ntp	FW
15	☒	on	passer	Network_in_vlan50_siem Network_in_vlan40_sauve	Resolveur	dns	FW

3. Création et préparation de la VM Debian sur Proxmox

- Déploiement d'une VM Debian 12.5 avec les caractéristiques suivantes :
 - Disque : 50 Go
 - Mémoire RAM : 8 Go
 - Processeurs : 2 Cores
 - Carte réseau connectée au bridge vrbr1 configuré avec le VLAN 60

4. Configuration réseau de la VM accueillant Docker

Paramètres réseau

La configuration réseau de la VM est la suivante :

- Adresse IP : 192.168.4.241
- Masque de sous-réseau : /29 (255.255.255.248)
- Réseau : 192.168.4.240
- Adresse de broadcast : 192.168.4.247
- Passerelle (gateway) : 192.168.4.246
- Nom de domaine : Dortmund.cub.private

Serveurs DNS

- DNS principal : 192.168.4.10 (resolveur)
- DNS secondaire : 9.9.9.9 (quad9)

Installation d'outils pratiques (SUR LA VM)

```
apt install numlockx net-tools sudo timeshift tree rsync apache2
```

Description des paquets :

- **numlockx**
Permet d'activer automatiquement le verrouillage numérique (Num Lock) au démarrage.
- **net-tools**
Fournit des outils réseau classiques comme ifconfig, netstat, route pour diagnostiquer et configurer le réseau.
- **sudo**
Permet d'exécuter des commandes avec les privilèges administrateur (super utilisateur).
- **timeshift**
Outil de sauvegarde et restauration du système (snapshots), pratique pour revenir en arrière après un problème.
- **tree**
Affiche l'arborescence des dossiers et fichiers sous forme de structure en arbre dans le terminal

- **rsync**
Permet de synchroniser efficacement des fichiers et des répertoires entre deux emplacements, en ne transférant que les différences. Idéal pour les sauvegardes et les déploiements.
- **apache2**
Serveur web permettant d'héberger des sites et des applications web, largement utilisé pour servir du contenu HTTP/HTTPS.

Installation de Docker (version récente – Debian) (SUR LA VM)

1. Mise à jour du système et installation des prérequis

Mettre à jour la liste des paquets et installer les outils nécessaires.

```
sudo apt update puis sudo apt install -y ca-certificates curl
```

Cette étape permet de préparer le système en installant les certificats de sécurité et l'outil curl utilisé pour télécharger des fichiers.

2. Création du répertoire de stockage des clés

Crée un dossier dédié pour stocker les clés de sécurité des dépôts.

```
sudo install -m 0755 -d /etc/apt/keyrings
```

Ce répertoire permet d'organiser les clés utilisées pour vérifier l'authenticité des paquets.

3. Ajout de la clé GPG officielle de Docker

Téléchargée la clé officielle de Docker afin de sécuriser l'installation des paquets.

```
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc  
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Cette clé garantit que les paquets installés proviennent bien de Docker.

4. Ajout du dépôt Docker

Ajouté le dépôt officiel Docker au système.

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF  
Types: deb  
URIs: https://download.docker.com/linux/debian  
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")  
Components: stable  
Signed-By: /etc/apt/keyrings/docker.asc  
EOF
```

Cette étape permet d'accéder aux versions récentes de Docker adaptées à ta version de Debian.

5. Mise à jour des paquets

Une mise à jour est nécessaire pour prendre en compte le nouveau dépôt.

```
sudo apt update
```

Le système peut désormais détecter les paquets Docker disponibles.

6. Installation de Docker

Docker et ses composants principaux sont installés.

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Cette commande installe le moteur Docker, l'interface en ligne de commande, ainsi que les outils nécessaires à la gestion des conteneurs.

7. Vérification de l'installation

Enfin, il faut vérifier que Docker est correctement installé.

```
docker --version
```

Source : <https://docs.docker.com/engine/install/debian/>

Installation de portainer.io (SUR LA VM)

1. Créer un volume pour Portainer avec la commande

```
docker volume create portainer_data
```

2. Pull l'image de portainer ce

```
docker pull portainer/portainer-ce:latest
```

3. Lancer le conteneur Portainer CE :

```
docker run -d -p 8000:8000 -p 9000:9000 --name portainer --restart=always -v /var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer-ce:latest
```

Le port **9000** correspond à l'interface principale, tandis que le port **8000** est utilisé pour la fonctionnalité **Edge** (facultative).

Il est également possible de publier sur le port **9443** afin de se connecter en **HTTPS**.

Pour vérifier que le conteneur fonctionne correctement, utilisez la commande : `docker ps`

Une fois le conteneur lancé, accédez à l'interface web via l'adresse : <http://localhost:9000>.

Source : <https://docs.portainer.io/start/install-ce/server/docker/linux>

Mise en service de HAproxy

1. Prérequis

- Avoir trois conteneurs (+/-) accueillants chacun un serveur web
- Mettre en place un réseau sur docker afin d'attribuer des adresses IPs fixe à nos trois serveurs web et à notre futur HAproxy.

2. Mise en place

1. Dans portainer.io, je crée un conteneur HAproxy :

- a. Je publie les ports 8080 en HOST et 80 en Container (frontend) et le port 8404 en HOST et 8404 en container (stats)

- b. Je créer un volume persistant entre ma machine local (Ma VM) et la CT pour insérer le futur fichier de configuration
- c. J'intègre HAproxy dans mon réseau docker avec une adresse fixe (10.10.10.10)

2. Création du fichier de configuration haproxy.cfg

```
GNU nano 7.2 haproxy.cfg *
global
    log stdout format raw local0
```

```
defaults
    log global
    mode http
    option httplog
    option httpchk GET /
    timeout connect 5s
    timeout client 30s
    timeout server 30s
```

```
frontend http_front
    bind *:80
    default_backend backends
```

```
backend backends
    balance roundrobin

    server nginx 10.10.10.20:80 check
    server nginx2 10.10.10.21:80 check
    server nginx3 10.10.10.22:80 check
```

Afin de fluidifier le trafic et d'améliorer la gestion des données, on crée un volume persistant relié à la machine locale. Celui-ci sera mis à jour toutes les cinq minutes.

```
bind *:8404
stats enable
stats uri /stats
stats refresh 10s
stats auth admin:admin
```

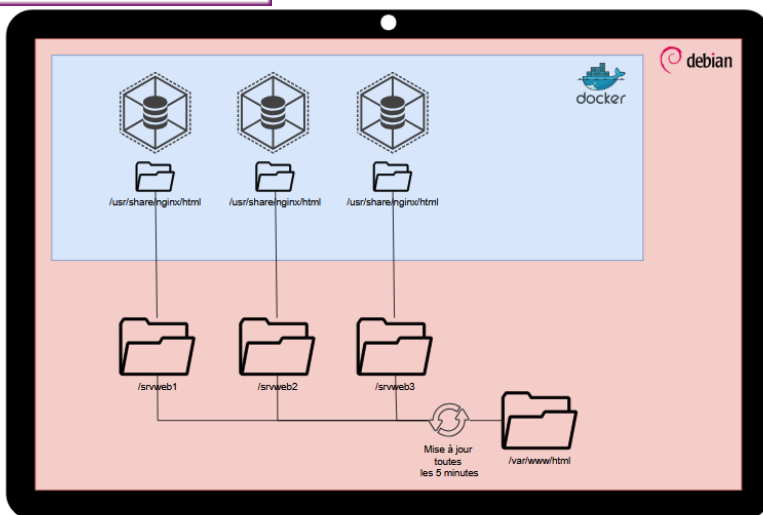
defaults définit les paramètres communs appliqués automatiquement à tous les frontends et backends. Elle permet d'éviter de répéter la configuration, comme le mode HTTP, les logs, les options de vérification (httpchk) et les délais (timeouts). C'est donc une base globale qui simplifie et uniformise le comportement de HAProxy.

Le frontend sert de porte d'entrée pour le trafic : c'est lui qui gère les demandes des clients. Il se charge d'écouter sur une adresse et un port spécifique (dans ce cas, le port 80) et détermine l'arrière-plan approprié pour diriger les requêtes. Des règles peuvent également être ajoutées pour filtrer ou diriger le trafic.

Le backend désigne l'ensemble des serveurs qui sont réellement chargés de traiter les demandes. HAProxy répartit la charge en utilisant une technique comme le round robin entre plusieurs serveurs, tels que les nginx. Il est également capable de contrôler leur statut afin d'éviter d'acheminer du trafic vers un serveur qui n'est pas disponible.

tion

Page statistique



1. Créer le script qui permet de copier le contenu de de /var/www/html dans /srvweb1 /srvweb2 et /srvweb3

```
# ! /bin/bash
rsync -a -delete /var/www/html/ /srvweb1
rsync -a -delete /var/www/html/ /srvweb2
rsync -a -delete /var/www/html/ /srvweb3
```

2. Le rendre exécutable

```
Chmod +x /.../synchro_web.sh
```

3. Créer une tâche automatique

```
Crontab -e
```

Sources : `* * * * * /.../synchro_web.sh`

Docker : <https://docs.docker.com/engine/install/debian/>

Portainer.io : <https://docs.portainer.io/start/install-ce/server/docker/linux>

HAproxy : <https://www.it-connect.fr/mise-en-place-dun-serveur-haproxy/>

<https://blog.stephane-robert.info/docs/services/reseau/haproxy/>

Rsync : <https://blog.stephane-robert.info/docs/admin-serveurs/linux/reseau/rsync/>

Cron : <https://blog.stephane-robert.info/docs/admin-serveurs/linux/exploiter/planification/cron/>